

Buenas prácticas en CSS

A.Valencia

0. Indenta bien tu código:

Utiliza 2 o 4 espacios para indentar tu CSS y HTML de forma consistente.

```
.column {  
  display: flex;  
  flex-direction: column;  
  gap: 10px;  
}
```

```
<header id="header">  
  <div class="div-fit div-fixed">  
    <h1>Fotoalbum Responsiu</h1>  
  </div>  
  <div class="div-fit div-hidden">  
    <h1>Fotoalbum Responsiu</h1>  
  </div>  
</header>
```

1. Sigue una convención de nombres por las clases:

Case type		Example
	Camel	thisNewVariable
	Pascal	ThisNewVariable
	Snake	this_new_variable
	Kebab	this-new-variable

2. Divide tu CSS en secciones:

```
/* Tipografía */ h1,  
h2, h3 {...}
```

```
/* Formularios */
```

```
...
```

```
/* Barra de navegación */
```

```
...
```

```
etc.
```

3. Evita selectores demasiado específicos:

~~div.nav ul.items li {...}~~ —————

4. Evita utilizar !importante

```
p
{ background-color: green !importante;
}
```

5. Ordena alfabéticamente las propiedades CSS:

```
.nav-item  
{ background-color: gray; color:  
  gold; fuente-  
  weight: bold; width:  
  100%; }
```

Para ordenar a VScode, selecciona el código, haz Ctrl+Mayús+P y “Sort Lines Ascending”.

6. Aprovecha la herencia de estilos:

Aprovecha que los elementos toman de su contenedor papá los estilos que no tienen definidos.

No vuelvas a definir los estilos por sus elementos hijos, cuando ya los heredan de su contenedor padre.

7. Extrae patrones repetitivos a clases reutilizables:

En vez de...

utiliza...

```
<div class="card">Card content</div>
<div class="alert">Alert content</div>
```

```
/* Repetitive styles */
.card {
  padding: 16px;
  border-radius: 8px;
  background-color: #f9f9f9;
}

.alert {
  padding: 16px;
  border-radius: 8px;
  background-color: #ffecec;
}
```

```
<div class="card padding-md border-radius-sm">Card content</div>
<div class="alert padding-md border-radius-sm">Alert content</div>

/* Reusable utility classes */
.padding-md {
  padding: 16px;
}

.border-radius-sm {
  border-radius: 8px;
}

/* Specific styles for components */
.card {
  background-color: #f9f9f9;
}

.alert {
  background-color: #ffecec;
}
```

8. Utiliza 'custom properties' cuando sea necesario:

CSS permite 'custom properties' (variables globales), que permiten cambiar su valor a un solo sitio del código y tener el efecto en todas partes

(La pseudo-clase `:root {...}` se refiere al elemento raíz `<html>`) `:root`

```
{ --color-primario: gold;

} div {
  color: var(--color-primario);
}
```

9. Utiliza un 'linter' (o un validador online de código):

Linter: Son herramientas de análisis de código que identifican problemas como errores de sintaxis, inconsistencias de estilo o posibles errores lógicos antes de ejecutar el código (análisis estático). Pueden ejecutarse en los editores mientras escribes el código.

Por CSS: [Stylelint \(demo\)](#). Por HTML: [HTMLHint \(demo\)](#). Por JS: [ESLint \(demo\)](#) _____

Validadores online:

Por CSS: [W3C](#). Por HTML: [W3C](#). Por JS: [ESLint online](#) _____

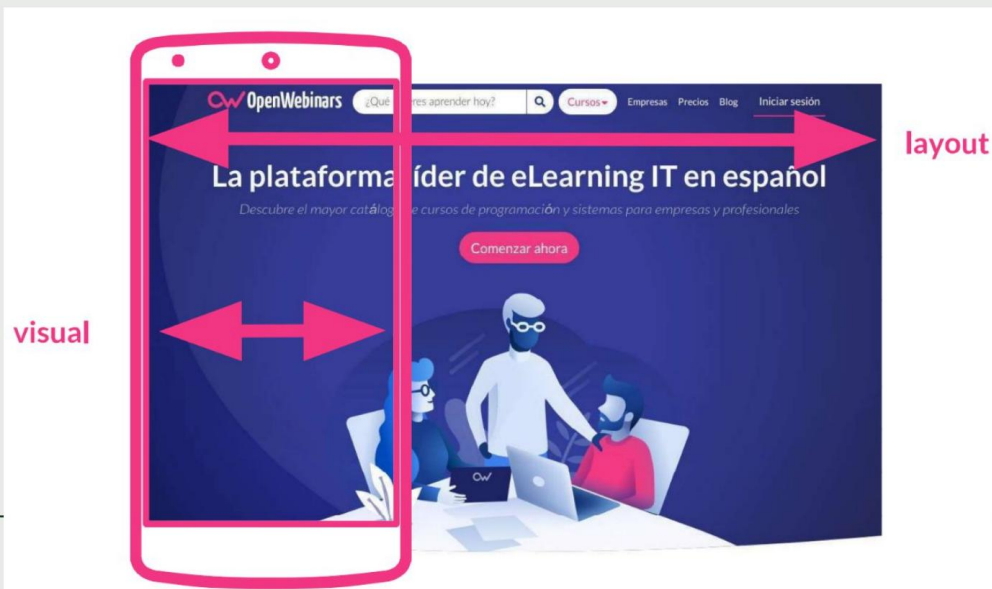
10. Viewport a dispositivos móviles:

En dispositivos móviles, unifica el 'layout-viewport' de CSS con el 'visual-viewport' de la pantalla:

```
<head>
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1"/>
```

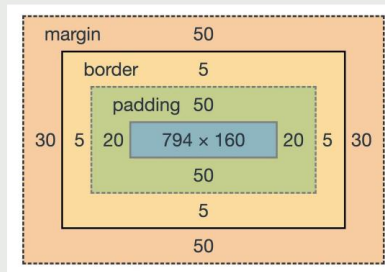
```
</head>
```



11. Simplifica los cálculos en el box-modelo:

Por defecto los elementos HTML se definen por width y height de la caja de contenido (caja en azul en la imagen), pero los cálculos (content+padding+border+margin) se simplifican si definimos los elementos por la caja de 'border' con ...

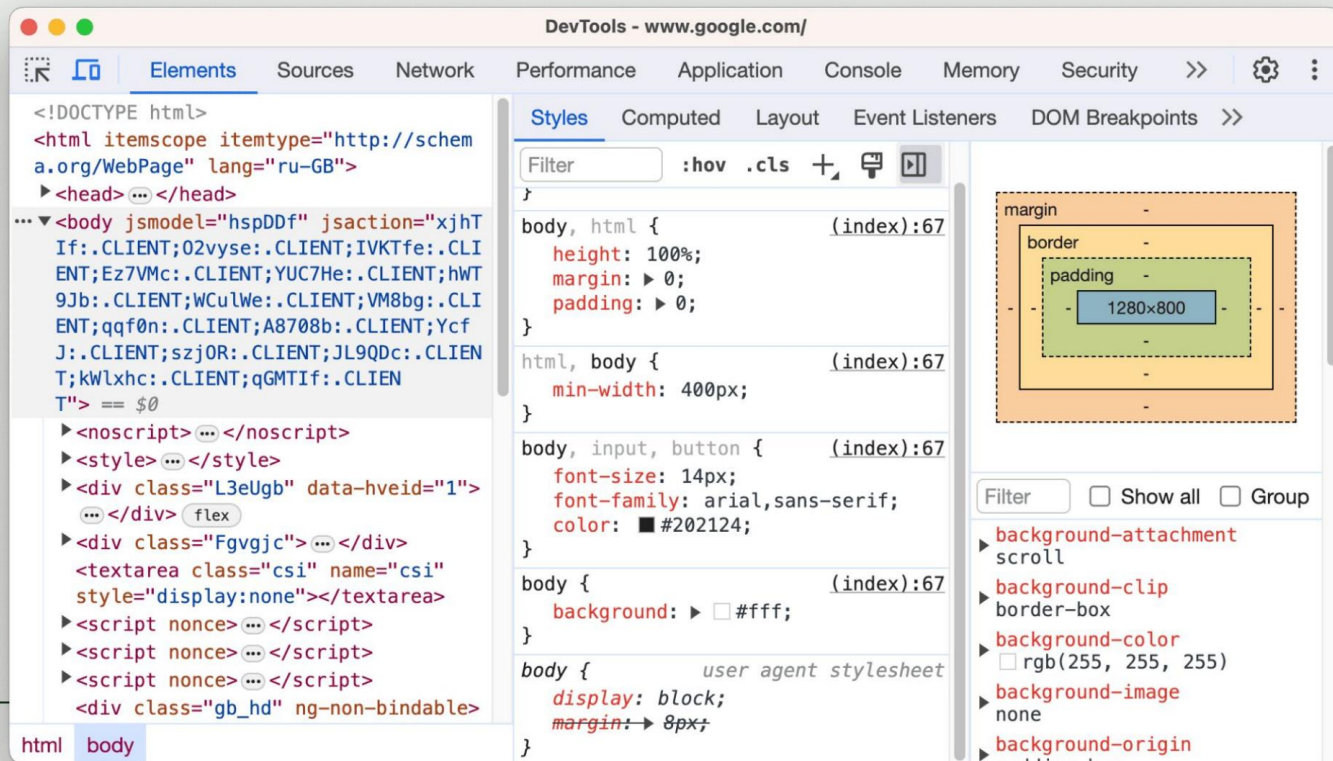
```
* {  
  box-sizing: border-box;  
}
```



Truco de maquetación: para visualizar todas las cajas (border-box), podemos...

```
* {  
  border: 1px sólido red;  
}
```

12. Utiliza las herramientas de desarrollador del navegador:



13. Utiliza Media Queries sólo cuando sea necesario:

A veces la función de las Media Queries pueden realizarla, según el caso,...

`flex-wrap: wrap;`

o

`height: fit-content;`

O...

14. Sigue los principios generales de la programación:

DRY: Don't Repeat Yourself (No repitas código)

KISS: Keep It Simple, Stupid! (No lo compliques)

15. Notación BEM (Block-Element-Modifier):

Block: Define un componente independiente y reutilizable de la interfaz.

Ejemplo: **menú**, **button**, **header**

Elemento: Son partes específicas de un blog, que dependen de éste para existir.

Ejemplo: **menu__item**, **button__icon**

Modifier: Son variaciones de un blog o elemento que cambian su aspecto o comportamiento.

Ejemplo: **menu__item--active**, **button--disabled**

```
<div class="button">
  <span class="button__icon">👤</span>
  <span class="button__text">Envia</span>
</div>
```

```
.button {
  display: flex;
  align-items: center;
}

.button__icon {
  margin-right: 5px;
}

.button__text {
  font-size: 16px;
}
```

16. En entornos de producción: optimización

Minificar el código CSS consiste en eliminar todos los elementos innecesarios para reducir su tamaño sin alterar su funcionalidad.

Online minifier: <https://www.toptal.com/developers/cssminifier>

CSS original:

```
body {  
    margin: 0;  
    padding: 0;  
}
```

CSS minificado:

```
body{margin:0;padding:0;}
```